Knowledge Representation for Network Change Approval

AIMLZG557 Artificial and Computational Intelligence · Situated Learning Assignment · Question 2 (individual) · Section B: Student Submission

B.1 Submission Form

Name	Chirag Bhatnagar
ID No	2025AG05066
Industry / Domain	Telecommunications: fixed-line fibre broadband (network automation)
Organisation Type	Internet service provider: Excitel Broadband, Gurugram office
[Instructor prompt]	Identify a real workplace process where decisions are governed by predefined rules.

B.2 Anchor (1 mark)

Workplace artifact: the change approval process that decides whether a configuration change may be pushed to the production network. I have worked as a network automation developer at Excitel Broadband (Gurugram office) for over three years, on the automation that generates, checks and pushes these changes.

Why it matters. Excitel is a fibre broadband provider. One wrong change on a core router or a BNG (broadband network gateway, which terminates subscriber sessions) can disconnect many home subscribers at once, and an outage in the evening peak is the most visible kind. So every change must pass the same written policy before it is pushed.

How a change flows:

1. An engineer raises a change request in the ticketing system: what changes, the target devices, the planned time slot and, for non-standard changes, a rollback plan.
2. The configuration is generated from an approved template (or written by hand) and reviewed by a second engineer.
3. Standard template changes on access devices are pre-approved. Every other change goes to the Change Advisory Board (CAB). Emergency changes also need the duty manager.
4. Before the push, the pipeline runs automatic gates: fresh config backup, golden-config compliance, device reachability, open incidents, maintenance window and change freeze.
5. The change is pushed. Post-checks compare the network before and after (for example the number of BGP peers), and a failed check triggers a rollback.

Who is involved: the requester, the peer reviewer, the CAB, the duty manager (emergencies only) and the automation pipeline.

Where the facts come from: the network inventory (device roles and regions), the ticketing system (changes, reviewers, approvals, schedule and freeze calendar), the config backup system, the compliance checker and the monitoring system (reachability, incidents, BGP peers).

Why a rule-based knowledge base fits: the policy is already a set of if–then rules, and every decision must be the same each time and explainable to an auditor.

Device names, people and change numbers in this document are illustrative. No production data is used.

B.3 Apply (2 marks)

Section B.3.1 lists the 12 business rules of the policy. B.3.2 states them in propositional logic and B.3.3 lifts them to predicate logic. B.3.4 builds a knowledge base of 76 facts and 20 inference rules from them, and B.3.5 runs five workplace queries against it.

B.3.1 Business rules

Rule	Business rule
BR1	A change that touches a core router or a BNG is high-risk.
BR2	A change built from a pre-approved template that touches only access devices (OLTs and aggregation switches) is a standard change.
BR3	A standard change is risk-cleared without further approval. Any other change is risk-cleared only with CAB approval and a written rollback plan.
BR4	An engineer other than the requester must review every change (four-eyes rule).
BR5	Changes run only in the night maintenance window (for example 01:00–05:00), never in the evening peak.
BR6	An emergency change approved by the duty manager may run outside the maintenance window.
BR7	During a change freeze in a region (for example the festive season), no change may touch devices in that region, except an emergency change approved by the duty manager.
BR8	Every target device must have a configuration backup taken in the last 24 hours.
BR9	The generated configuration must pass the golden-config checks: Telnet disabled, NTP configured and TACACS+ login configured.
BR10	Every target device must be reachable and have no open P1 incident.
BR11	A change that passes BR3 to BR10 is approved for push. An approved high-risk change is pushed with an engineer on call.
BR12	After a push, if a target device has fewer BGP peers than before the change, the change must be rolled back.

B.3.2 Propositional logic

Each proposition is a statement about the one change under review, and it is either true or false.

Symbol	Meaning	Symbol	Meaning
CR	The change touches a core router	**FO**	The change is clear of any change freeze
BN	The change touches a BNG	**BK**	Every target device has a backup from the last 24 hours
HR	The change is high-risk	**BO**	The backup check passes
TP	The change is built from a pre-approved template	**NT**	Telnet is disabled in the generated config
AO	Every device the change touches is an access device	**NP**	NTP is configured in the generated config
ST	The change is a standard change	**TC**	TACACS+ login is configured in the generated config
CA	The CAB has approved the change	**CO**	The generated config is compliant
RP	The change has a written rollback plan	**RE**	Every target device is reachable
RC	The change is risk-cleared	**NI**	No target device has an open P1 incident
RV	The reviewer is a different engineer from the requester	**PC**	The pre-checks pass
PR	The change is peer-reviewed	**AP**	The change is approved for push
NW	The change is scheduled in the night window	**OC**	An engineer must be on call during the push
EM	The change is an emergency change	**DP**	The change has been pushed
DM	The duty manager has approved the change	**BG**	A target device has fewer BGP peers after the push than before
TO	The change's timing is allowed	**RR**	The change must be rolled back
FZ	A device the change touches is in a frozen region		

The 12 business rules in propositional logic:

Rule	Propositional formula
BR1	$(CR \vee BN) \rightarrow HR$
BR2	$(TP \wedge AO) \rightarrow ST$
BR3	$ST \rightarrow RC$ $(CA \wedge RP) \rightarrow RC$
BR4	$RV \rightarrow PR$
BR5	$NW \rightarrow TO$
BR6	$(EM \wedge DM) \rightarrow TO$
BR7	$\neg FZ \rightarrow FO$ $(EM \wedge DM) \rightarrow FO$
BR8	$BK \rightarrow BO$
BR9	$(NT \wedge NP \wedge TC) \rightarrow CO$
BR10	$(RE \wedge NI) \rightarrow PC$
BR11	$(RC \wedge PR \wedge TO \wedge FO \wedge BO \wedge CO \wedge PC) \rightarrow AP$ $(AP \wedge HR) \rightarrow OC$
BR12	$(DP \wedge BG) \rightarrow RR$

BR5 says changes run **only** in the window. BR5 and BR6 are the only rules that make TO true, so a change scheduled in the evening peak without emergency approval never gets TO, and therefore never gets AP.

B.3.3 Predicate logic

The same rules extended with constants, variables, predicates and quantifiers.

Constants

Type	Constants
Changes	CHG100, CHG101, CHG102, CHG103, CHG104, CHG105
Devices	ggn-cr-01, ggn-cr-02, ggn-agg-07, ggn-olt-12, dli-bng-02, dli-olt-05
Device roles	CoreRouter, BNG, OLT, AggSwitch
Regions	Gurugram, Delhi
Engineers	Asha, Ravi, Meera, Karan
Time slots	NightWindow, EveningPeak
Check phases	Before, After

Variables: c a change, d a device, r a region, x and y engineers, s a time slot, $n1$ and $n2$ numbers of BGP peers.

Predicates

Predicate	Meaning	Kind
$Role(d, role)$	Device d has this role	Fact
$InRegion(d, r)$	Device d is in region r	Fact
$Open(r)$	Region r has no change freeze	Fact
$Targets(c, d)$	Change c touches device d	Fact
$FromTemplate(c)$	Change c is built from a pre-approved template	Fact
$RequestedBy(c, x)$	Engineer x raised change c	Fact
$ReviewedBy(c, y)$	Engineer y reviewed change c	Fact
$CABApproved(c)$	The CAB approved change c	Fact
$RollbackPlan(c)$	Change c has a written rollback plan	Fact
$Emergency(c)$	Change c is an emergency change	Fact
$DutyManagerApproved(c)$	The duty manager approved change c	Fact
$ScheduledIn(c, s)$	Change c is scheduled in time slot s	Fact
$BackupFresh(d)$	Device d has a config backup from the last 24 hours	Fact
$NoTelnet(c)$, $NtpSet(c)$, $TacacsSet(c)$	The config of change c passes that golden-config check	Fact
$Reachable(d)$	Device d answers the reachability check	Fact

Predicate	Meaning	Kind
IncidentFree(d)	Device d has no open P1 incident	Fact
Deployed(c)	Change c has been pushed	Fact
BgpPeers(d, phase, n)	Device d had n BGP peers before or after the push	Fact
CoreDevice(d), AccessDevice(d)	Device d is a core device (core router, BNG) or an access device (OLT, aggregation switch)	Derived
HighRisk(c), Standard(c)	Change c is high-risk, or a standard change	Derived
RiskCleared(c), PeerReviewed(c)	Change c passes the approval or four-eyes rule	Derived
TimeOK(c), FreezeOK(c)	Change c passes the timing or freeze rule	Derived
BackupOK(c), Compliant(c)	Change c passes the backup or compliance rule	Derived
Healthy(d), PrecheckOK(c)	Device d is healthy; every device of change c is healthy	Derived
Approved(c), EngineerOnCall(c)	Change c may be pushed; it needs an engineer on call	Derived
RollbackRequired(c)	Change c must be rolled back	Derived

Rules in predicate logic

Rule	Predicate logic formula
BR1	$\forall d\ (\text{Role}(d, \text{CoreRouter}) \vee \text{Role}(d, \text{BNG}) \rightarrow \text{CoreDevice}(d))$ $\forall c\ \forall d\ (\text{Targets}(c, d) \wedge \text{CoreDevice}(d) \rightarrow \text{HighRisk}(c))$
BR2	$\forall d\ (\text{Role}(d, \text{OLT}) \vee \text{Role}(d, \text{AggSwitch}) \rightarrow \text{AccessDevice}(d))$ $\forall c\ (\text{FromTemplate}(c) \wedge \forall d\ (\text{Targets}(c, d) \rightarrow \text{AccessDevice}(d)) \rightarrow \text{Standard}(c))$
BR3	$\forall c\ (\text{Standard}(c) \rightarrow \text{RiskCleared}(c))$ $\forall c\ (\text{CABApproved}(c) \wedge \text{RollbackPlan}(c) \rightarrow \text{RiskCleared}(c))$
BR4	$\forall c\ \forall x\ \forall y\ (\text{RequestedBy}(c, x) \wedge \text{ReviewedBy}(c, y) \wedge x \neq y \rightarrow \text{PeerReviewed}(c))$
BR5	$\forall c\ (\text{ScheduledIn}(c, \text{NightWindow}) \rightarrow \text{TimeOK}(c))$
BR6	$\forall c\ (\text{Emergency}(c) \wedge \text{DutyManagerApproved}(c) \rightarrow \text{TimeOK}(c))$
BR7	$\forall c\ (\forall d\ (\text{Targets}(c, d) \rightarrow \exists r\ (\text{InRegion}(d, r) \wedge \text{Open}(r))) \rightarrow \text{FreezeOK}(c))$ $\forall c\ (\text{Emergency}(c) \wedge \text{DutyManagerApproved}(c) \rightarrow \text{FreezeOK}(c))$
BR8	$\forall c\ (\forall d\ (\text{Targets}(c, d) \rightarrow \text{BackupFresh}(d)) \rightarrow \text{BackupOK}(c))$
BR9	$\forall c\ (\text{NoTelnet}(c) \wedge \text{NtpSet}(c) \wedge \text{TacacsSet}(c) \rightarrow \text{Compliant}(c))$
BR10	$\forall d\ (\text{Reachable}(d) \wedge \text{IncidentFree}(d) \rightarrow \text{Healthy}(d))$ $\forall c\ (\forall d\ (\text{Targets}(c, d) \rightarrow \text{Healthy}(d)) \rightarrow \text{PrecheckOK}(c))$
BR11	$\forall c\ (\text{RiskCleared}(c) \wedge \text{PeerReviewed}(c) \wedge \text{TimeOK}(c) \wedge \text{FreezeOK}(c) \wedge \text{BackupOK}(c) \wedge \text{Compliant}(c) \wedge \text{PrecheckOK}(c) \rightarrow \text{Approved}(c))$ $\forall c\ (\text{Approved}(c) \wedge \text{HighRisk}(c) \rightarrow \text{EngineerOnCall}(c))$
BR12	$\forall c\ \forall d\ \forall n1\ \forall n2\ (\text{Deployed}(c) \wedge \text{Targets}(c, d) \wedge \text{BgpPeers}(d, \text{Before}, n1) \wedge \text{BgpPeers}(d, \text{After}, n2) \wedge n2 < n1 \rightarrow \text{RollbackRequired}(c))$

How the quantifiers are used:

- $\forall$ over c and d makes one rule cover every change and every device.
- A nested $\forall d\ (\text{Targets}(c, d) \rightarrow \ldots)$ inside a condition (BR2, BR7, BR8, BR10) means "every device this change touches".
- $\exists r$ in BR7 means "the device sits in some region that is open".
- BR4 uses equality ($x \neq y$) and BR12 a numeric comparison ($n2 < n1$).
- BR7 uses a positive fact $\text{Open}(r)$ instead of $\neg\text{Frozen}(r)$, so no rule needs negation in its conditions. That keeps every rule usable for both forward and backward chaining.

B.3.4 Knowledge base

The knowledge base holds **76 facts** and **20 inference rules**. Appendix A has the same knowledge base as a Python program. I ran it and compared its output with each answer in B.3.5.

Changes in the knowledge base

Change	What it is	Target devices	Region
CHG100	Routing policy change, pushed last night	ggn-cr-02 (core router)	Gurugram
CHG101	Add a customer VLAN from the VLAN template	ggn-olt-12 (OLT), ggn-agg-07 (aggregation switch)	Gurugram
CHG102	BGP policy change	ggn-cr-01 (core router)	Gurugram
CHG103	Port profile update from the OLT template	dli-olt-05 (OLT)	Delhi (frozen)
CHG104	ACL update from the ACL template	ggn-agg-07 (aggregation switch)	Gurugram
CHG105	Emergency fix on a BNG	dli-bng-02 (BNG)	Delhi (frozen)

Facts (76), in groups:

Facts	Group and facts	Count
F1–F6	**Device roles:** Role(ggn-cr-01, CoreRouter), Role(ggn-cr-02, CoreRouter), Role(ggn-agg-07, AggSwitch), Role(ggn-olt-12, OLT), Role(dli-bng-02, BNG), Role(dli-olt-05, OLT)	6
F7–F13	**Regions:** InRegion(d, Gurugram) for ggn-cr-01, ggn-cr-02, ggn-agg-07, ggn-olt-12; InRegion(d, Delhi) for dli-bng-02, dli-olt-05; Open(Gurugram). Delhi has no Open fact because it is under a festive change freeze.	7
F14–F20	**Targets:** Targets(CHG100, ggn-cr-02), Targets(CHG101, ggn-olt-12), Targets(CHG101, ggn-agg-07), Targets(CHG102, ggn-cr-01), Targets(CHG103, dli-olt-05), Targets(CHG104, ggn-agg-07), Targets(CHG105, dli-bng-02)	7
F21–F23	**Templates:** FromTemplate(CHG101), FromTemplate(CHG103), FromTemplate(CHG104)	3
F24–F33	**People:** RequestedBy(CHG101, Asha), ReviewedBy(CHG101, Ravi), RequestedBy(CHG102, Ravi), ReviewedBy(CHG102, Meera), RequestedBy(CHG103, Karan), ReviewedBy(CHG103, Asha), RequestedBy(CHG104, Meera), ReviewedBy(CHG104, Meera), RequestedBy(CHG105, Karan), ReviewedBy(CHG105, Ravi)	10
F34–F38	**Approvals:** RollbackPlan(CHG102), CABApproved(CHG105), RollbackPlan(CHG105), Emergency(CHG105), DutyManagerApproved(CHG105)	5
F39–F43	**Schedule:** ScheduledIn(c, NightWindow) for CHG101, CHG102, CHG103, CHG104; ScheduledIn(CHG105, EveningPeak)	5
F44–F48	**Backups:** BackupFresh(d) for ggn-cr-01, ggn-agg-07, ggn-olt-12, dli-bng-02, dli-olt-05	5
F49–F63	**Compliance:** NoTelnet(c), NtpSet(c), TacacsSet(c) for each change CHG101 to CHG105	15
F64–F73	**Health:** Reachable(d) and IncidentFree(d) for the same five devices as the backups	10
F74–F76	**Post-check:** Deployed(CHG100), BgpPeers(ggn-cr-02, Before, 4), BgpPeers(ggn-cr-02, After, 3)	3
	Total	**76**

Inference rules (20). Every variable is universally quantified and every rule has one conclusion. Sixteen are definite clauses; K6, K12, K14 and K17 also test all targets of a change.

Rule	Inference rule	From
K1	Role(d, CoreRouter) → CoreDevice(d)	BR1
K2	Role(d, BNG) → CoreDevice(d)	BR1
K3	Role(d, OLT) → AccessDevice(d)	BR2
K4	Role(d, AggSwitch) → AccessDevice(d)	BR2
K5	Targets(c, d) ∧ CoreDevice(d) → HighRisk(c)	BR1
K6	FromTemplate(c) ∧ ∀d (Targets(c, d) → AccessDevice(d)) → Standard(c)	BR2
K7	Standard(c) → RiskCleared(c)	BR3
K8	CABApproved(c) ∧ RollbackPlan(c) → RiskCleared(c)	BR3
K9	RequestedBy(c, x) ∧ ReviewedBy(c, y) ∧ x ≠ y → PeerReviewed(c)	BR4
K10	ScheduledIn(c, NightWindow) → TimeOK(c)	BR5
K11	Emergency(c) ∧ DutyManagerApproved(c) → TimeOK(c)	BR6
K12	∀d (Targets(c, d) → ∃r (InRegion(d, r) ∧ Open(r))) → FreezeOK(c)	BR7
K13	Emergency(c) ∧ DutyManagerApproved(c) → FreezeOK(c)	BR7
K14	∀d (Targets(c, d) → BackupFresh(d)) → BackupOK(c)	BR8
K15	NoTelnet(c) ∧ NtpSet(c) ∧ TacacsSet(c) → Compliant(c)	BR9
K16	Reachable(d) ∧ IncidentFree(d) → Healthy(d)	BR10
K17	∀d (Targets(c, d) → Healthy(d)) → PrecheckOK(c)	BR10
K18	RiskCleared(c) ∧ PeerReviewed(c) ∧ TimeOK(c) ∧ FreezeOK(c) ∧ BackupOK(c) ∧ Compliant(c) ∧ PrecheckOK(c) → Approved(c)	BR11

Rule	Inference rule	From
K19	$\text{Approved}(c) \land \text{HighRisk}(c) \rightarrow \text{EngineerOnCall}(c)$	BR11
K20	$\text{Deployed}(c) \land \text{Targets}(c, d) \land \text{BgpPeers}(d, \text{Before}, n1) \land \text{BgpPeers}(d, \text{After}, n2) \land n2 < n1 \rightarrow \text{RollbackRequired}(c)$	BR12

K6, K12, K14 and K17 check every Targets fact of the change, so they treat the list of target devices as complete. In Prolog these four rules would use forall/2.

B.3.5 Workplace queries

Five questions the team asks before a maintenance night. Query 1 and Query 4 use forward chaining (start from the facts), Query 2 uses backward chaining (start from the goal), Query 3 asks for all answers to a query with a variable, and Query 5 uses resolution.

Query 1: Is CHG101 approved for tonight's maintenance window?

Method: forward chaining. Rules fire on known facts until $\text{Approved}(\text{CHG101})$ appears.

Step	New fact	Rule	Because of
1	AccessDevice(ggn-olt-12)	K3	Role(ggn-olt-12, OLT)
2	AccessDevice(ggn-agg-07)	K4	Role(ggn-agg-07, AggSwitch)
3	Standard(CHG101)	K6	FromTemplate(CHG101); both targets are access devices (steps 1 and 2)
4	RiskCleared(CHG101)	K7	step 3
5	PeerReviewed(CHG101)	K9	RequestedBy(CHG101, Asha), ReviewedBy(CHG101, Ravi), Asha ≠ Ravi
6	TimeOK(CHG101)	K10	ScheduledIn(CHG101, NightWindow)
7	FreezeOK(CHG101)	K12	both targets are in Gurugram, and Open(Gurugram)
8	BackupOK(CHG101)	K14	BackupFresh(ggn-olt-12), BackupFresh(ggn-agg-07)
9	Compliant(CHG101)	K15	NoTelnet, NtpSet, TacacsSet for CHG101
10	Healthy(ggn-olt-12), Healthy(ggn-agg-07)	K16	Reachable and IncidentFree for both devices
11	PrecheckOK(CHG101)	K17	step 10
12	Approved(CHG101)	K18	steps 4 to 9 and step 11

Answer: yes. $\text{HighRisk}(\text{CHG101})$ cannot be derived because neither target is a core device, so K19 does not fire: the pipeline pushes CHG101 in tonight's window without an engineer on call.

Query 2: Can CHG102 (BGP policy change on core router ggn-cr-01) be pushed tonight?

Method: backward chaining. Start from the goal $\text{Approved}(\text{CHG102})$ and work back to facts.

Step	Goal	How it is resolved	Result
1	Approved(CHG102)	Only K18 concludes Approved, so its seven conditions become subgoals.	7 subgoals
2	PeerReviewed(CHG102)	K9 with x = Ravi, y = Meera, and Ravi ≠ Meera	✓ proved
3	TimeOK(CHG102)	K10 with ScheduledIn(CHG102, NightWindow)	✓ proved
4	FreezeOK, BackupOK, Compliant, PrecheckOK	K12, K14, K15, K17: ggn-cr-01 is in open Gurugram, has a fresh backup, passes the three checks and is healthy	✓ proved
5	RiskCleared(CHG102) by K7	Needs Standard(CHG102), which by K6 needs FromTemplate(CHG102). No such fact and no rule concludes it. (ggn-cr-01 is not an access device either.)	✗ fails
6	RiskCleared(CHG102) by K8	Needs CABApproved(CHG102). No such fact and no rule concludes it. RollbackPlan(CHG102) holds, but it is not enough on its own.	✗ fails
7	Approved(CHG102)	Subgoal RiskCleared(CHG102) cannot be proved	✗ fails

Answer: no. The change is blocked until the CAB approves it. The KB can also prove $\text{HighRisk}(\text{CHG102})$ (K1, K5), so once the CAB approves, K19 will also require an engineer on call.

Query 3: Which changes are approved for push tonight?

Method: the query $\exists c\ \mathrm{Approved}(c)$ is answered by finding every value of c that makes it true. The table shows each condition of K18 for the five changes under review.

Change	Risk cleared	Peer reviewed	Time OK	Freeze OK	Backup OK	Compliant	Pre-check OK	Approved
CHG101	✓	✓	✓	✓	✓	✓	✓	✓ **yes**
CHG102	✗	✓	✓	✓	✓	✓	✓	✗ no
CHG103	✓	✓	✓	✗	✓	✓	✓	✗ no
CHG104	✓	✗	✓	✓	✓	✓	✓	✗ no
CHG105	✓	✓	✓	✓	✓	✓	✓	✓ **yes**

Answer: $c = \mathrm{CHG101}$ and $c = \mathrm{CHG105}$. The other three are blocked for one reason each:

- **CHG102:** no CAB approval, so $\mathrm{RiskCleared}$ fails (Query 2).
- **CHG103:** it touches dli-olt-05 in Delhi, which has no Open fact, and it is not an emergency, so neither K12 nor K13 gives $\mathrm{FreezeOK}$. It is blocked even though it is a standard change.
- **CHG104:** Meera both raised and reviewed it, so $x \neq y$ fails in K9 and $\mathrm{PeerReviewed}$ fails.

CHG100 is not in the table because it was already pushed last night (Query 5).

Query 4: Can emergency change CHG105 run on dli-bng-02 tonight, outside the window and during the Delhi freeze?

Method: forward chaining, following the emergency exception rules.

Step	New fact	Rule	Because of
1	$\mathrm{TimeOK}(\mathrm{CHG105})$	K11	$\mathrm{Emergency}(\mathrm{CHG105})$, $\mathrm{DutyManagerApproved}(\mathrm{CHG105})$. K10 does not apply: it is scheduled in the evening peak.
2	$\mathrm{FreezeOK}(\mathrm{CHG105})$	K13	The same two facts. K12 does not apply: Delhi has no Open fact.
3	$\mathrm{RiskCleared}(\mathrm{CHG105})$	K8	$\mathrm{CABApproved}(\mathrm{CHG105})$, $\mathrm{RollbackPlan}(\mathrm{CHG105})$. It is not a standard change: no template, and a BNG is not an access device.
4	$\mathrm{PeerReviewed}(\mathrm{CHG105})$	K9	$\mathrm{RequestedBy}(\mathrm{CHG105}, \mathrm{Karan})$, $\mathrm{ReviewedBy}(\mathrm{CHG105}, \mathrm{Ravi})$, $\mathrm{Karan} \neq \mathrm{Ravi}$
5	$\mathrm{BackupOK}(\mathrm{CHG105})$, $\mathrm{Compliant}(\mathrm{CHG105})$	K14, K15	$\mathrm{BackupFresh}(\mathrm{dli\text{-}bng\text{-}02})$; all three golden-config checks pass
6	$\mathrm{Healthy}(\mathrm{dli\text{-}bng\text{-}02})$, $\mathrm{PrecheckOK}(\mathrm{CHG105})$	K16, K17	$\mathrm{Reachable}(\mathrm{dli\text{-}bng\text{-}02})$, $\mathrm{IncidentFree}(\mathrm{dli\text{-}bng\text{-}02})$
7	$\mathrm{Approved}(\mathrm{CHG105})$	K18	steps 1 to 6
8	$\mathrm{CoreDevice}(\mathrm{dli\text{-}bng\text{-}02})$, $\mathrm{HighRisk}(\mathrm{CHG105})$	K2, K5	$\mathrm{Role}(\mathrm{dli\text{-}bng\text{-}02}, \mathrm{BNG})$, $\mathrm{Targets}(\mathrm{CHG105}, \mathrm{dli\text{-}bng\text{-}02})$
9	$\mathrm{EngineerOnCall}(\mathrm{CHG105})$	K19	steps 7 and 8

Answer: yes, with an engineer on call. The emergency exception lifts the window and freeze rules, but every safety gate (review, CAB, backup, compliance, health) still applies. CHG103 is in the same frozen region but has no emergency approval, so it stays blocked.

Query 5: Must CHG100 be rolled back after last night's push?

Method: resolution refutation. Write K20 as a clause, add the negated goal, and resolve until the empty clause appears.

K20 as a clause: $\neg\mathrm{Deployed}(c) \vee \neg\mathrm{Targets}(c, d) \vee \neg\mathrm{BgpPeers}(d, \mathrm{Before}, n1) \vee \neg\mathrm{BgpPeers}(d, \mathrm{After}, n2) \vee \neg\mathrm{Less}(n2, n1) \vee \mathrm{RollbackRequired}(c)$

Negated goal: $\neg\mathrm{RollbackRequired}(\mathrm{CHG100})$

Step	Resolve with	Substitution	Resolvent
1	Negated goal and K20	c / CHG100	¬Deployed(CHG100) ∨ ¬Targets(CHG100, d) ∨ ¬BgpPeers(d, Before, n1) ∨ ¬BgpPeers(d, After, n2) ∨ ¬Less(n2, n1)
2	Deployed(CHG100)	none	¬Targets(CHG100, d) ∨ ¬BgpPeers(d, Before, n1) ∨ ¬BgpPeers(d, After, n2) ∨ ¬Less(n2, n1)
3	Targets(CHG100, ggn-cr-02)	d / ggn-cr-02	¬BgpPeers(ggn-cr-02, Before, n1) ∨ ¬BgpPeers(ggn-cr-02, After, n2) ∨ ¬Less(n2, n1)
4	BgpPeers(ggn-cr-02, Before, 4)	n1 / 4	¬BgpPeers(ggn-cr-02, After, n2) ∨ ¬Less(n2, 4)
5	BgpPeers(ggn-cr-02, After, 3)	n2 / 3	¬Less(3, 4)
6	Less(3, 4), true by arithmetic	none	□ (empty clause)

Answer: yes. The empty clause shows the negated goal contradicts the knowledge base, so $KB \models \text{RollbackRequired}(\text{CHG100})$. ggn-cr-02 lost a BGP peer (4 before, 3 after), and the pipeline rolls the change back.

B.4 Analyse (1 mark)

How predicate logic improves on propositional logic here

With predicate logic, the same 20 rules decide every change on every device. With propositional logic, each change needs its own copy of the 31 symbols, and the "every device", "different engineer" and "fewer peers" conditions cannot be checked inside the logic at all.

What the workplace needs	Propositional logic	Predicate logic
Many changes and devices	Symbols describe one change. Five changes need five copies of all 31 symbols and 12 rules; every new change or device needs new symbols.	One rule covers all changes through variables. A new change or device is just new facts.
"Every device the change touches" (BR2, BR7, BR8, BR10)	Hidden inside one symbol (AO, BK, RE, NI). Something outside the logic must check each device.	$\forall d\ (\text{Targets}(c, d) \rightarrow \ldots)$ checks each device inside the KB. Query 1 checked both CHG101 targets.
"A different engineer" (BR4)	RV is simply asserted as true or false.	$x \neq y$ is checked. Query 3 caught Meera reviewing her own change, CHG104.
"In an open region" (BR7)	FZ is asserted.	$\exists r\ (\text{InRegion}(d, r) \wedge \text{Open}(r))$ links each device to its region.
Numbers (BR12)	BG is asserted.	$n2 < n1$ compares the BGP peer counts (Query 5).
Open questions	Only yes or no about one fixed change.	A query with a variable returns answers: $\exists c$ Approved(c) gave CHG101 and CHG105.

How well the knowledge base solves the problem

All five queries came out the way the policy intends, and each answer can be traced back through the rules and facts that produced it.

- It gives the same decision every time, with a reason the requester can act on: CHG102 needs CAB approval, CHG104 needs a different reviewer.
- The chain of rules can be stored with the change ticket as audit evidence.
- It catches what people miss under time pressure: a self-review, or a change that touches a frozen region.

Missing knowledge and limitations

- **Closed world.** A missing fact counts as false. That is safe when it blocks a change (no backup fact, no push), but stale facts are risky. Delhi counts as frozen only because $\text{Open}(\text{Delhi})$ is absent; if the freeze calendar is not synced, an old $\text{Open}(\text{Delhi})$ fact would let changes through.

- **No topology.** The KB does not know that an aggregation switch depends on a core router, or that two changes on the same path on the same night interact. It cannot estimate how many subscribers a change could affect.
- **Yes-or-no risk.** A change is high-risk or not. There is no degree of risk based on subscriber count, the device's history or the size of the change.
- **Coarse time.** Time slots and "backup in the last 24 hours" arrive as facts from other systems. The KB cannot reason about actual timestamps or how long a change will take.
- **Conclusions are never withdrawn.** If a P1 incident opens after $\mathrm{Approved}(\mathrm{CHG101})$ is derived, the KB must be rebuilt before the push, or it will act on an old answer.
- **Thin post-checks.** Only BGP peer counts are compared. Subscriber session counts, interface errors and traffic levels are missing.

B.5 Reflect (1 mark)

Practical constraints on deployment

- **Incomplete and stale knowledge.** The facts come from five systems (inventory, ticketing, backups, compliance, monitoring). Each can lag behind or disagree with the live network.
- **Evolving business rules.** CAB policy, freeze calendars and golden-config checks change during the year. The rules need a named owner, version control and a test per rule, as code does. Without that, the KB keeps enforcing a policy that is out of date.
- **Privacy and security.** The KB holds device inventory, topology and staff names. Access must be limited and logged, credentials and subscriber data must never enter it, and staff data falls under India's Digital Personal Data Protection Act, 2023.
- **Scalability.** An ISP network has thousands of devices and many changes each night. Re-deriving every fact for every change will not scale; a rule engine that updates incrementally (Prolog, Datalog, or a policy engine such as Open Policy Agent) would.
- **Integration and trust.** The KB has to plug into the ticketing system and the pipeline, explain each block in plain words, and allow an audited human override in emergencies. Otherwise engineers will work around it.

The single most important improvement

Feed the knowledge base live, time-stamped facts, and block any change whose facts are missing or out of date.

Queries 1 to 5 show that the rules give the right answer when the facts are right. So in deployment the biggest risk is wrong facts, not wrong logic: an $\mathrm{Open}(\mathrm{Delhi})$ fact left over from before the freeze, or a $\mathrm{BackupFresh}$ fact for a backup that has since failed.

Each fact should be pulled from its source system just before the push, stamped with its time, and given a maximum age (for example 24 hours for backups and a few minutes for reachability). If a needed fact is missing or too old, the gate fails closed and the change waits. A delayed change costs one night; a wrong approval can cost an evening-peak outage. The emergency path (BR6, BR7) still lets urgent fixes through with the duty manager's approval.

This also fixes the closed-world limit from B.4: a missing fact then means "unknown, so stop", not "false, so go".

What I learned

- Turning the policy into logic made me pin down terms that were loose. "Standard change" had to become "built from a template and touches only access devices".

- It exposed cases a written policy can leave open, such as who approves a non-template change on an access device. In my KB it is not standard, so it falls to the CAB through K8; in a real deployment the policy owner would need to confirm that.
- Using positive facts like $\mathrm{Open}(r)$ instead of negation kept every rule usable for both forward and backward chaining. The price is that safety now depends on the facts being right, which is why the improvement above matters most.
- In a real deployment the facts would come from APIs instead of being typed in, the rules would live in a version-controlled repository with tests, and every decision would be logged against its change ticket.

Appendix A: Python implementation

I coded the knowledge base in Python 3 using only the standard library (file change_approval_kb.py). Forward chaining applies the 20 rules until no new fact appears; each derived fact records the rule and premises that produced it, which gives the explanations in B.3.5. The run derived 54 new facts.

Rule-firing code (rules K1 to K20):

```
def fire(kb):
    """Yield (conclusion, rule, premises) for each rule whose premises hold."""
    has = kb.__contains__
    for fact in list(kb):
        kind = fact[0]
        if kind == "Role":
            _, d, role = fact
            rule = {"CoreRouter": "K1", "BNG": "K2", "OLT": "K3", "AggSwitch": "K4"}[role]
            cls = "CoreDevice" if role in ("CoreRouter", "BNG") else "AccessDevice"
            yield (cls, d), rule, [fact]
        elif kind == "Targets":
            _, c, d = fact
            if has(("CoreDevice", d)):
                yield ("HighRisk", c), "K5", [fact, ("CoreDevice", d)]
        elif kind == "Reachable":
            d = fact[1]
            if has(("IncidentFree", d)):
                yield ("Healthy", d), "K16", [fact, ("IncidentFree", d)]
        elif kind == "Deployed":
            c = fact[1]
            for d in targets(kb, c):
                before = [f[3] for f in kb if f[:3] == ("BgpPeers", d, "Before")]
                after = [f[3] for f in kb if f[:3] == ("BgpPeers", d, "After")]
                for n1 in before:
                    for n2 in after:
                        if n2 < n1:
                            yield ("RollbackRequired", c), "K20", [
                                fact, ("Targets", c, d), ("BgpPeers", d, "Before", n1),
                                ("BgpPeers", d, "After", n2), f"{n2} < {n1}"]

    for c in sorted({f[1] for f in kb if f[0] == "Targets"}):
        ts = targets(kb, c)
        if has(("FromTemplate", c)) and all(has(("AccessDevice", d)) for d in ts):
            yield ("Standard", c), "K6", [("FromTemplate", c)] + [("AccessDevice", d) for d in ts]
        if has(("Standard", c)):
            yield ("RiskCleared", c), "K7", [("Standard", c)]
        if has(("CABApproved", c)) and has(("RollbackPlan", c)):
            yield ("RiskCleared", c), "K8", [("CABApproved", c), ("RollbackPlan", c)]
        for x in people(kb, "RequestedBy", c):
            for y in people(kb, "ReviewedBy", c):
                if x != y:
                    yield ("PeerReviewed", c), "K9", [("RequestedBy", c, x), ("ReviewedBy", c, y), f"{x} ≠ {y}"]
        if has(("ScheduledIn", c, "NightWindow")):
            yield ("TimeOK", c), "K10", [("ScheduledIn", c, "NightWindow")]
        if has(("Emergency", c)) and has(("DutyManagerApproved", c)):
            yield ("TimeOK", c), "K11", [("Emergency", c), ("DutyManagerApproved", c)]
            yield ("FreezeOK", c), "K13", [("Emergency", c), ("DutyManagerApproved", c)]
        open_regions = []
        for d in ts:
            regions = [f[2] for f in kb if f[:2] == ("InRegion", d) and has(("Open", f[2]))]
            if not regions:
                break
            open_regions += [("InRegion", d, regions[0]), ("Open", regions[0])]
        else:
            yield ("FreezeOK", c), "K12", list(dict.fromkeys(open_regions))
        if all(has(("BackupFresh", d)) for d in ts):
```

```
            yield ("BackupOK", c), "K14", [("BackupFresh", d) for d in ts]
        checks = [("NoTelnet", c), ("NtpSet", c), ("TacacsSet", c)]
        if all(map(has, checks)):
            yield ("Compliant", c), "K15", checks
        if all(has(("Healthy", d)) for d in ts):
            yield ("PrecheckOK", c), "K17", [("Healthy", d) for d in ts]
        if all(has((g, c)) for g in GATES):
            yield ("Approved", c), "K18", [(g, c) for g in GATES]
        if has(("Approved", c)) and has(("HighRisk", c)):
            yield ("EngineerOnCall", c), "K19", [("Approved", c), ("HighRisk", c)]
```

The program running. The screenshots below were taken in a Windows console with Python 3.10. The command is python change_approval_kb.py followed by the query numbers to run. Each tree shows a derived fact with its rule in brackets, and the facts or comparisons it was derived from indented below it.

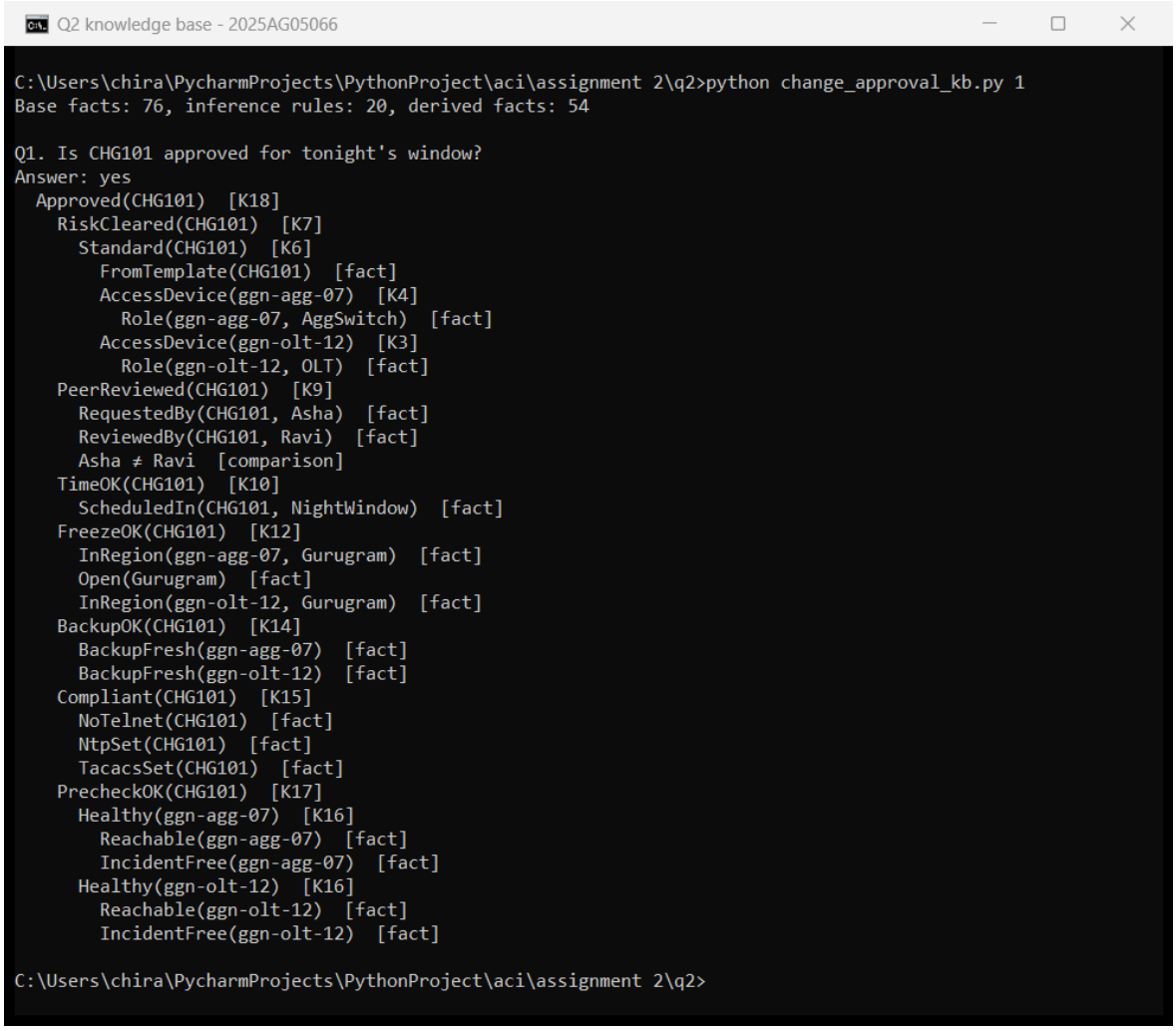


Figure A1: Query 1, the full derivation of Approved(CHG101).

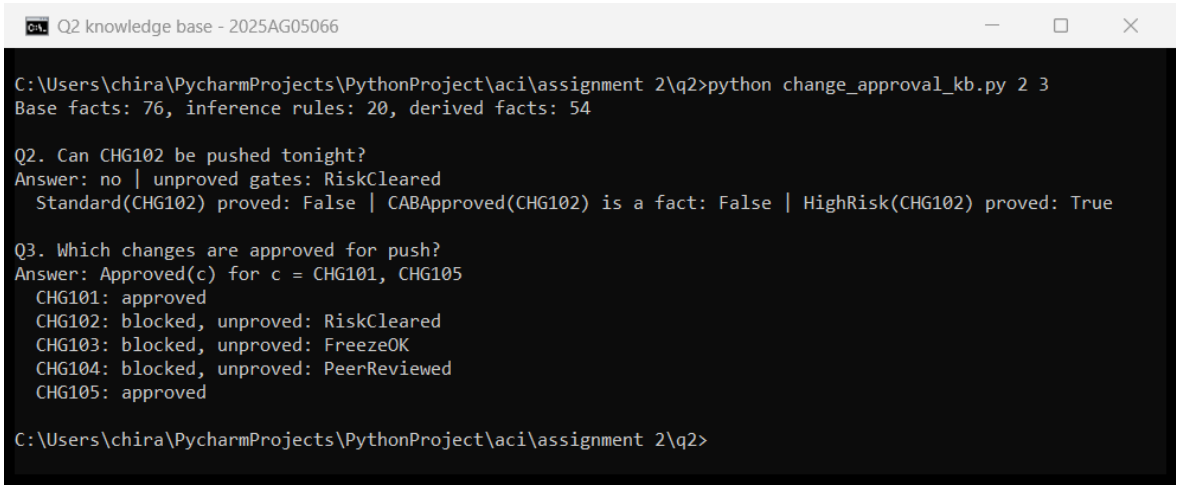


Figure A2: Query 2 (CHG102 blocked at RiskCleared) and Query 3 (all approved changes).

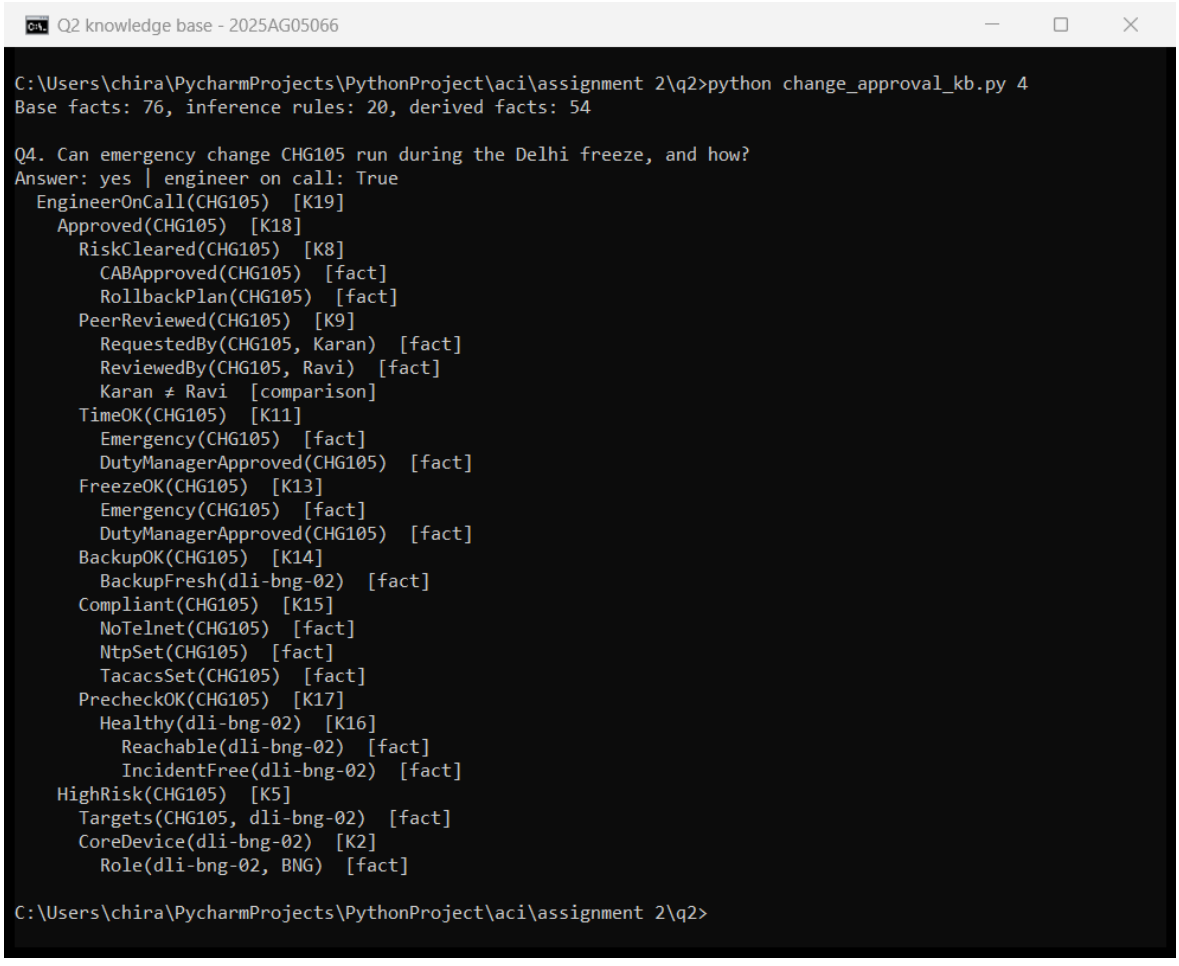


Figure A3: Query 4, the emergency change CHG105 approved with an engineer on call.

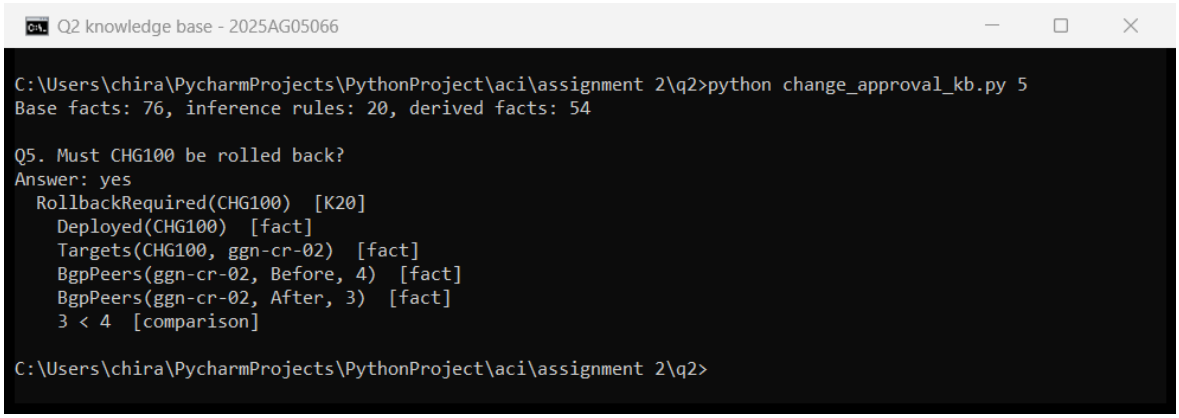


Figure A4: Query 5, the rollback of CHG100 after its BGP peers fell from 4 to 3.